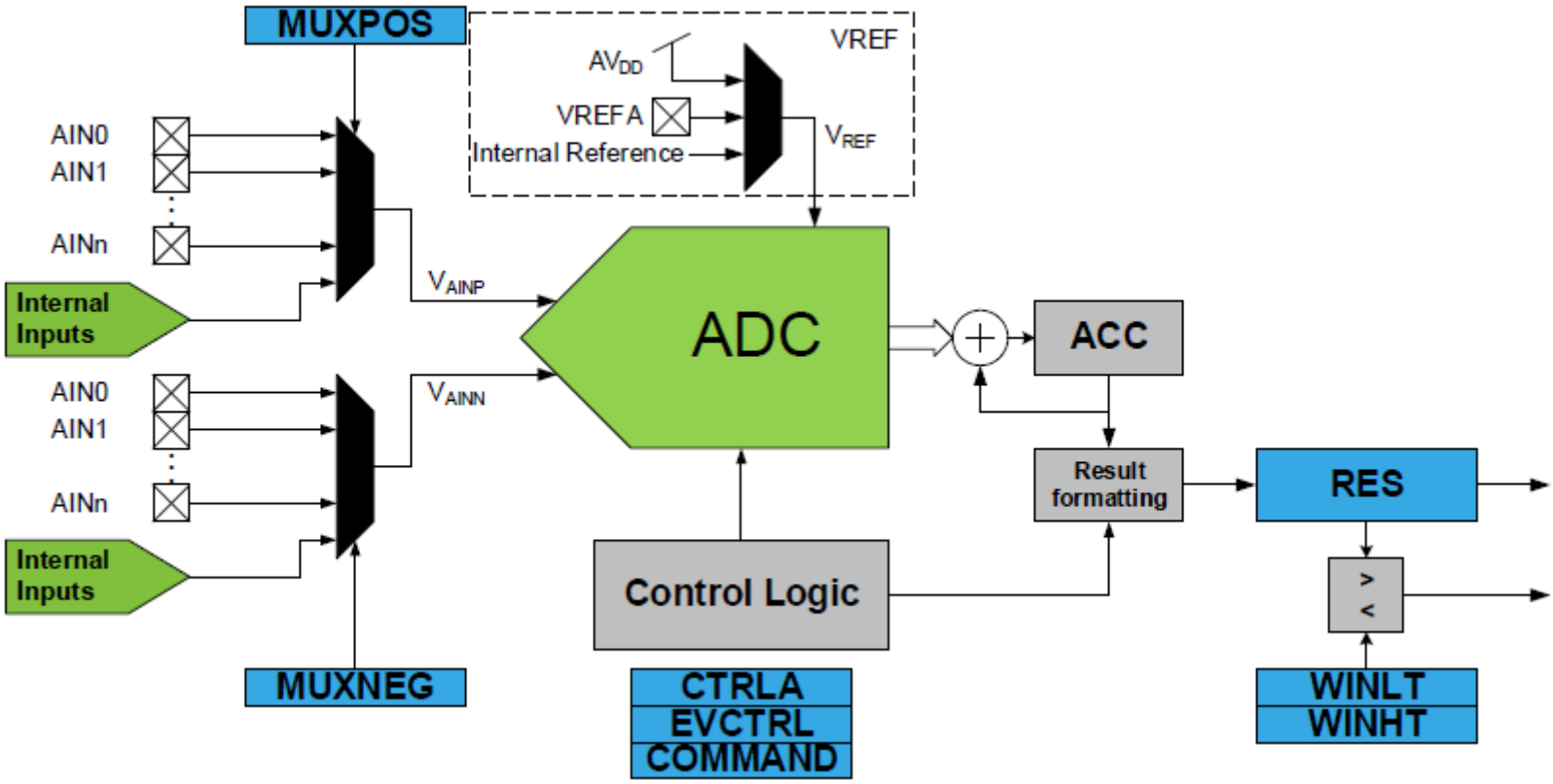
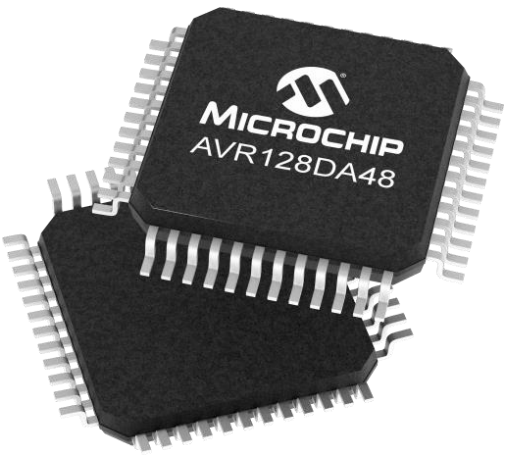
“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 18 - ADC Practical Sampling, Averaging, and Hardware Triggers





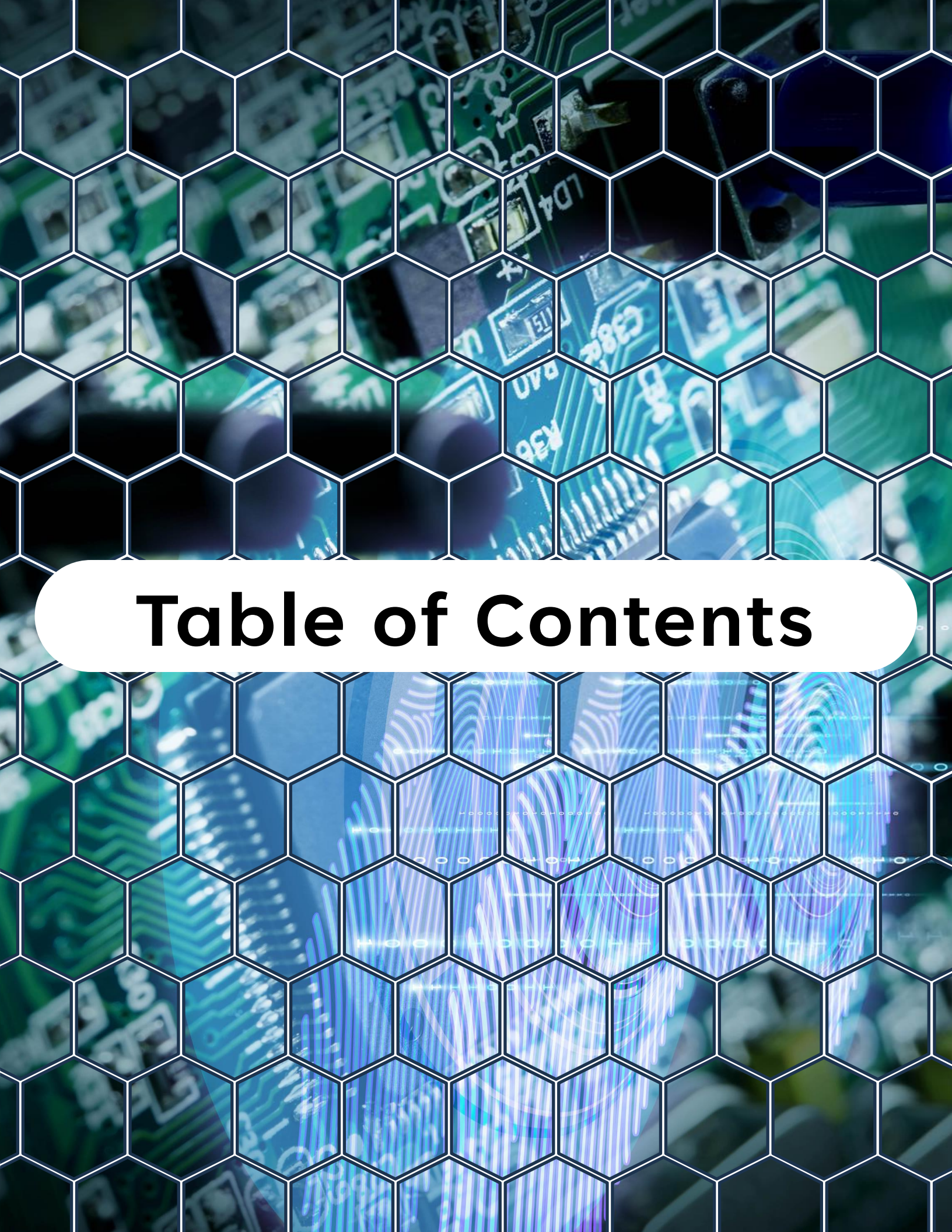
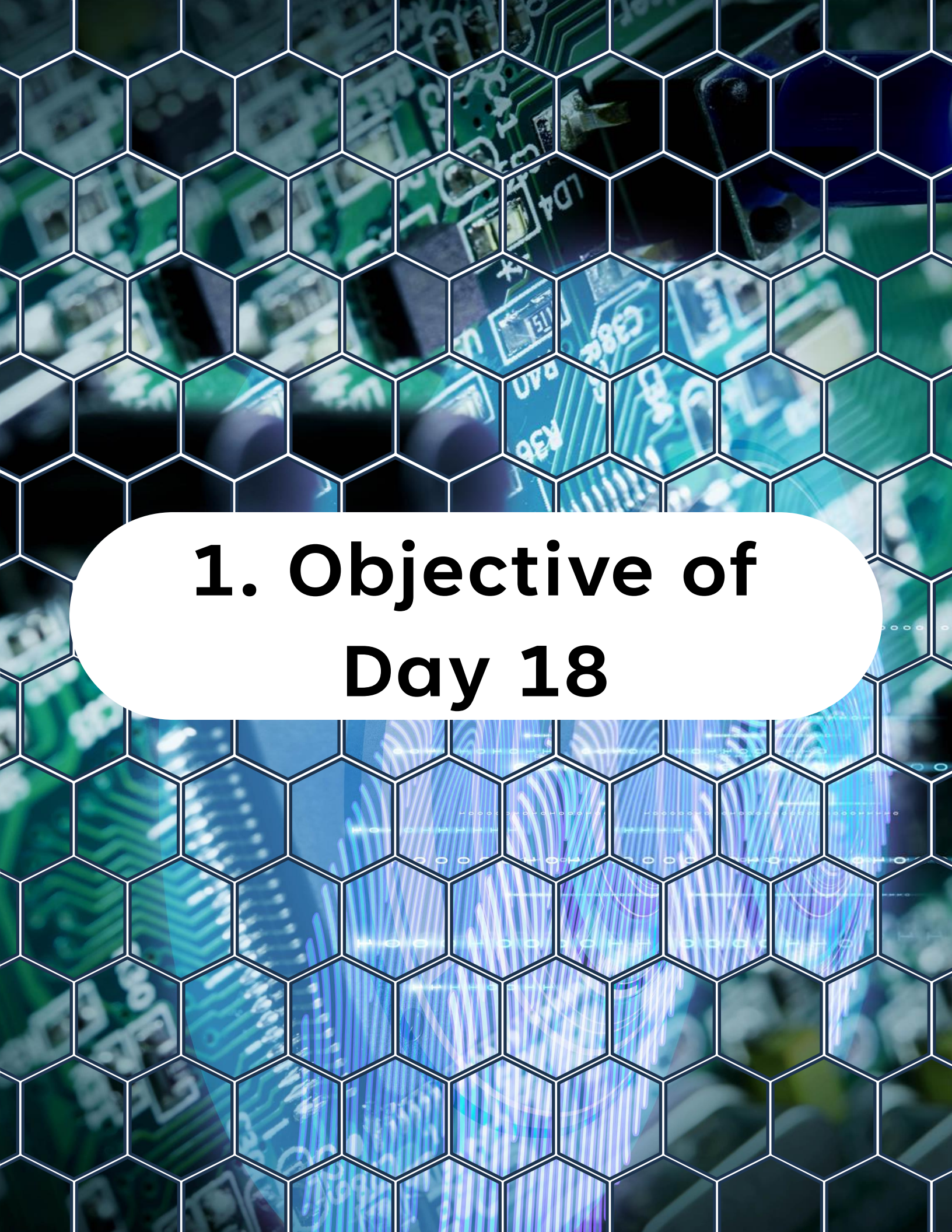


Table of Contents

Table of Contents

Table of Contents



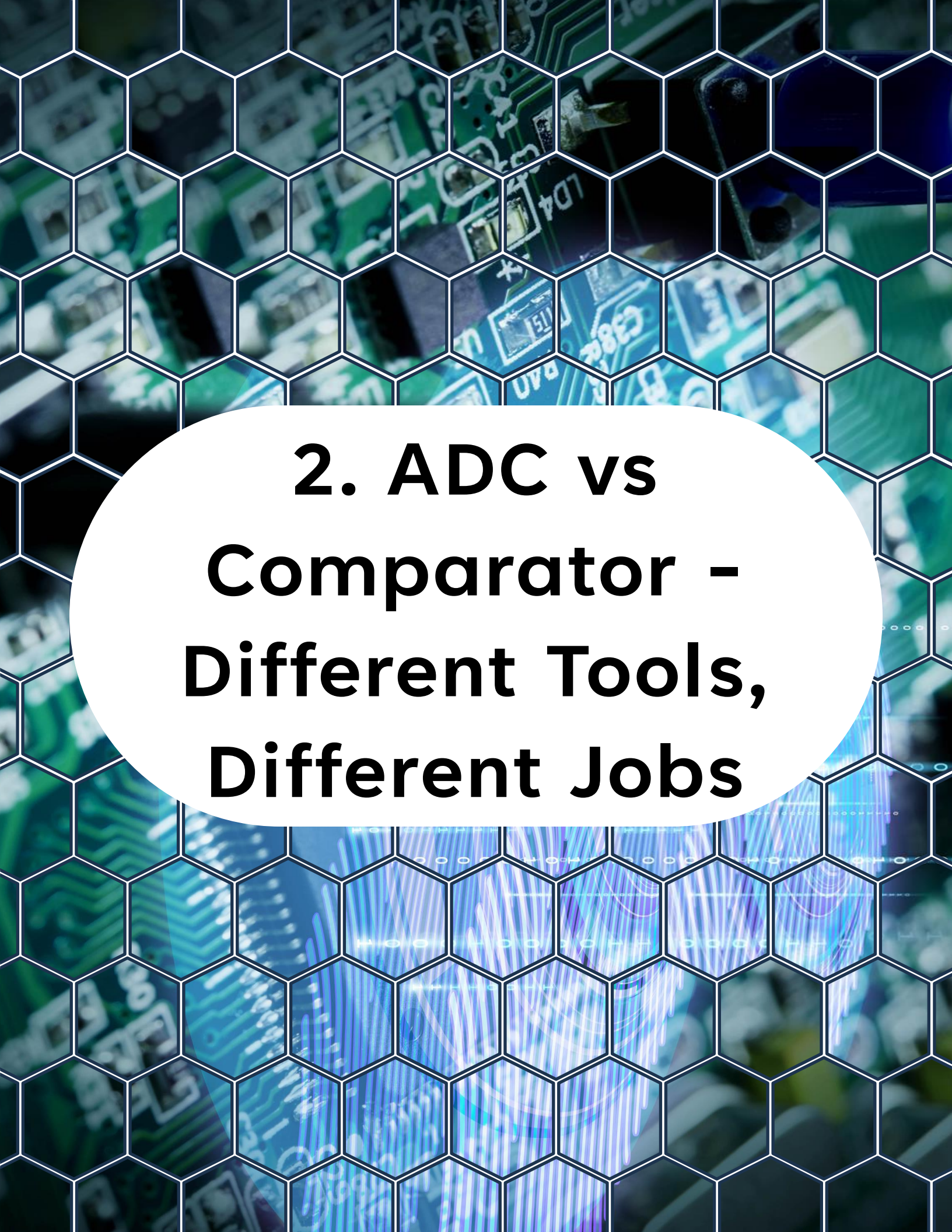
1. Objective of Day 18

1. Objective of Day 18

Today you move from threshold detection (AC) to real measurement. You will learn how to:

- Configure the **ADC** on AVR128DA48
- Perform single-shot and continuous conversions
- Apply averaging and oversampling techniques
- Reduce noise in real hardware
- Trigger ADC conversions using hardware (timers and EVSYS)
- Design non-blocking ADC sampling patterns

By the end of this lesson, you will be able to build robust sensor acquisition systems instead of just "reading a value".



2. ADC vs Comparator – Different Tools, Different Jobs

2. ADC vs Comparator - Different Tools, Different Jobs

Yesterday (Day 17), the Analog Comparator answered:

"Did the signal cross a threshold?“

Today, the ADC answers:

"What is the actual voltage?“

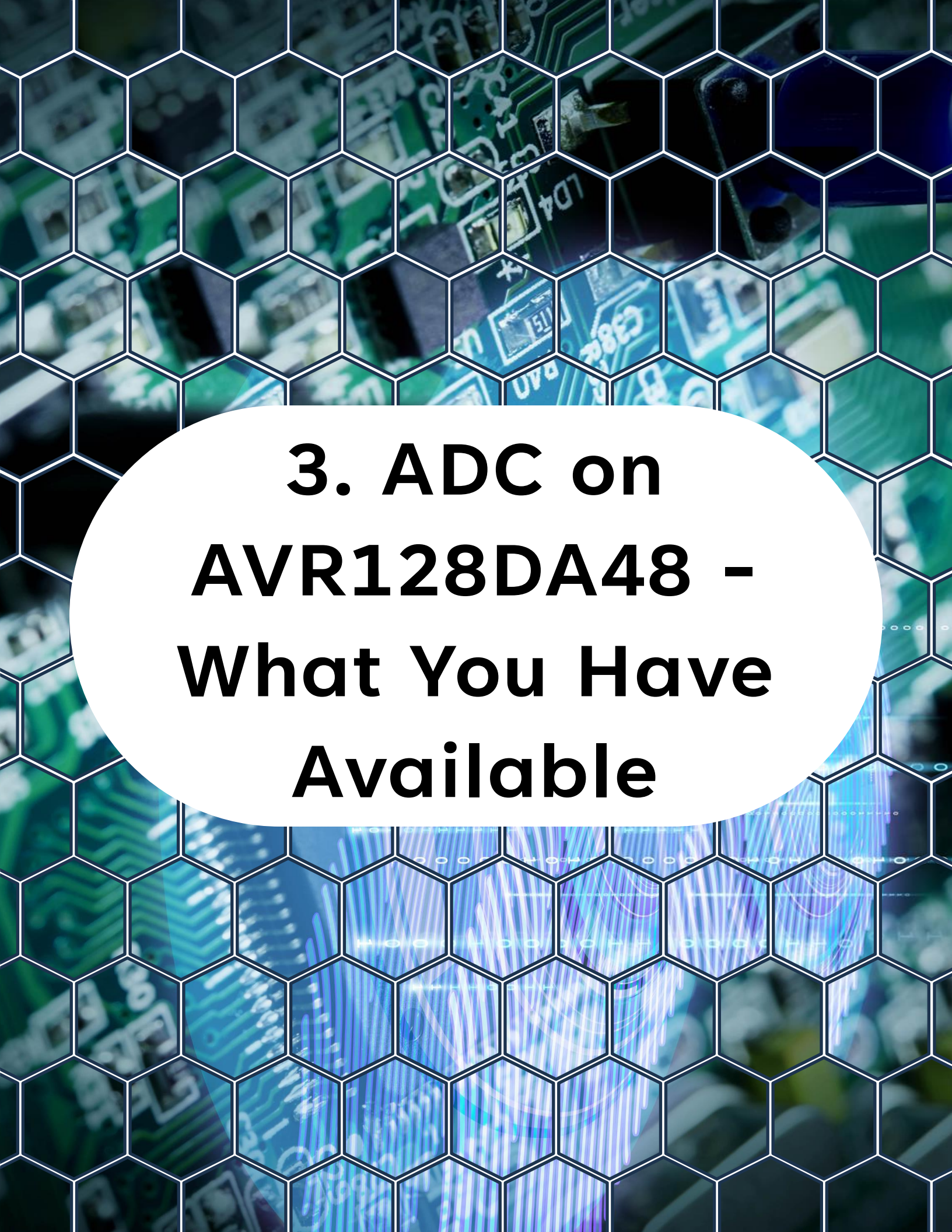
Comparator:

- FastBinary output
- Ideal for protection and timing

ADC:

- SlowerMulti-bit resolution (10-bit or more)
- Ideal for sensors and measurements

Both often work together in real products.



3. ADC on AVR128DA48 – What You Have Available

3. ADC on AVR128DA48 - What You Have Available

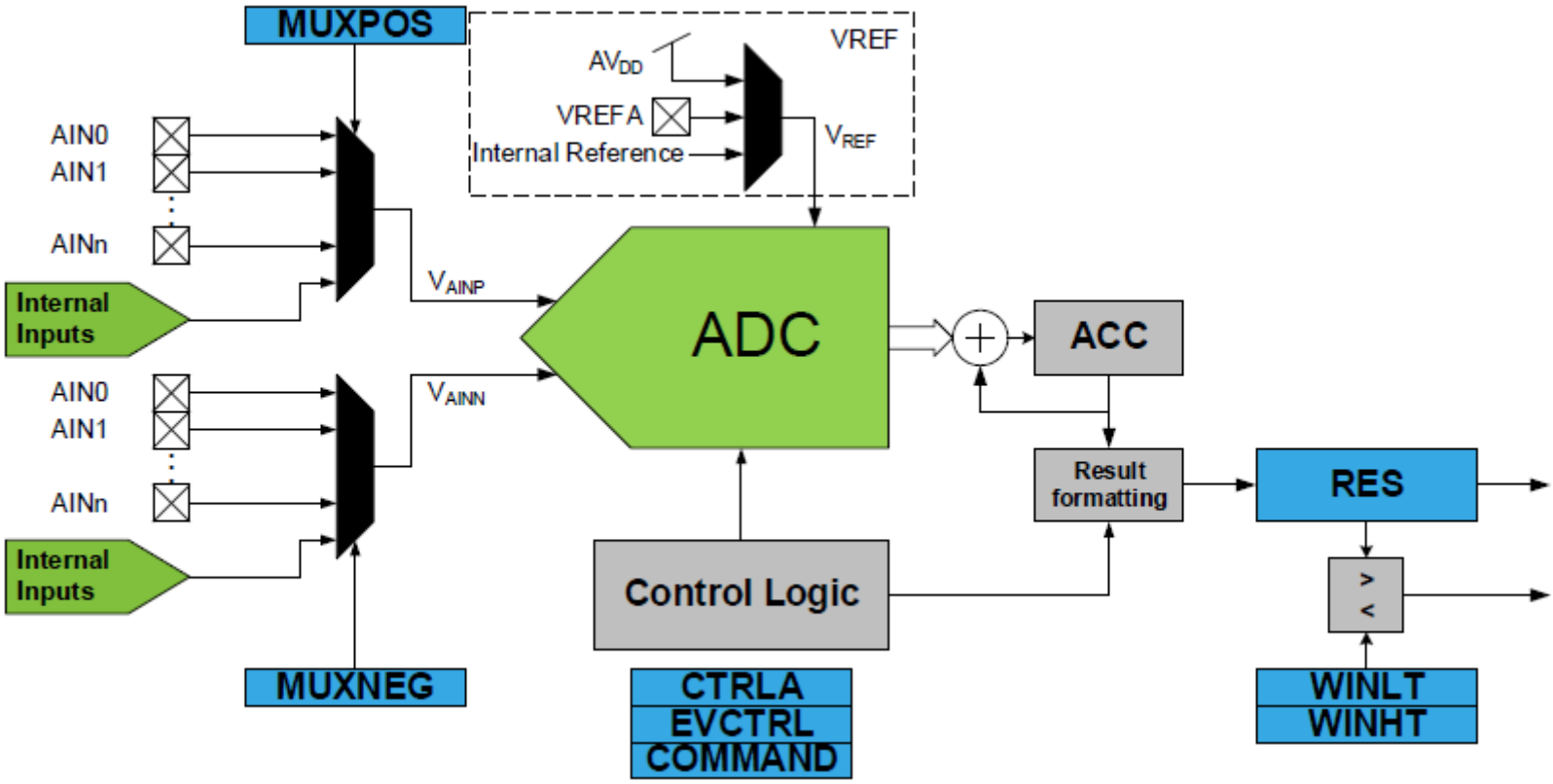
The AVR DA ADC typically provides:

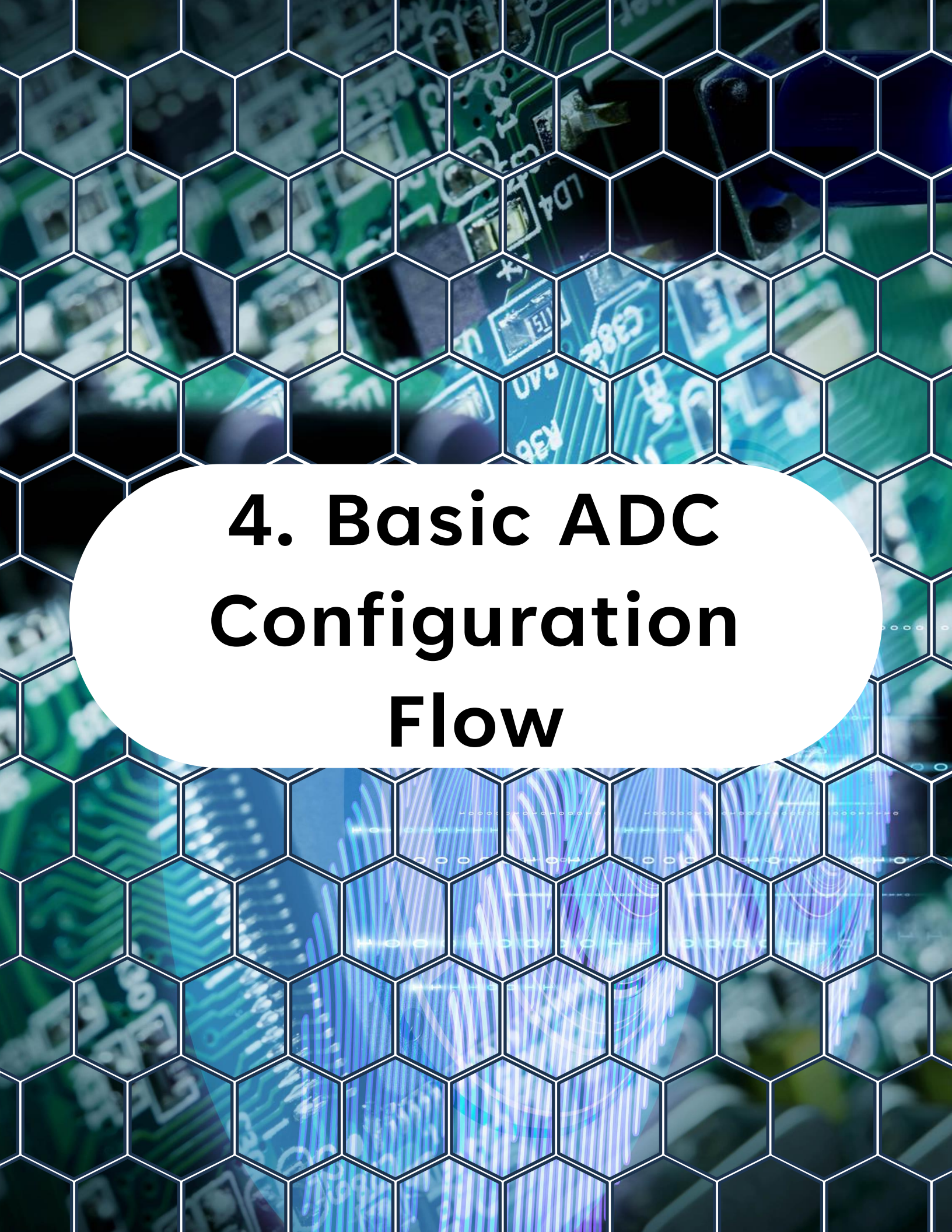
- 10-bit resolution
- Multiple input channels (MUX selectable)
- Internal references (1.1V, VDD, etc.)
- Accumulation / hardware averaging
- Event-triggered conversions
- Interrupt support
- Free-running mode

This is not just a "start conversion and wait" peripheral. It is a configurable measurement engine.

3. ADC on AVR128DA48 - What You Have Available

Block Diagram





4. Basic ADC Configuration Flow

4. Basic ADC Configuration Flow

A minimal single-ended ADC setup includes:

- Select voltage reference (VREF)
- Select input channel
- Set prescaler (ADC clock)
- Enable ADC
- Start conversion
- Wait for result (poll or interrupt)

Conceptual example:

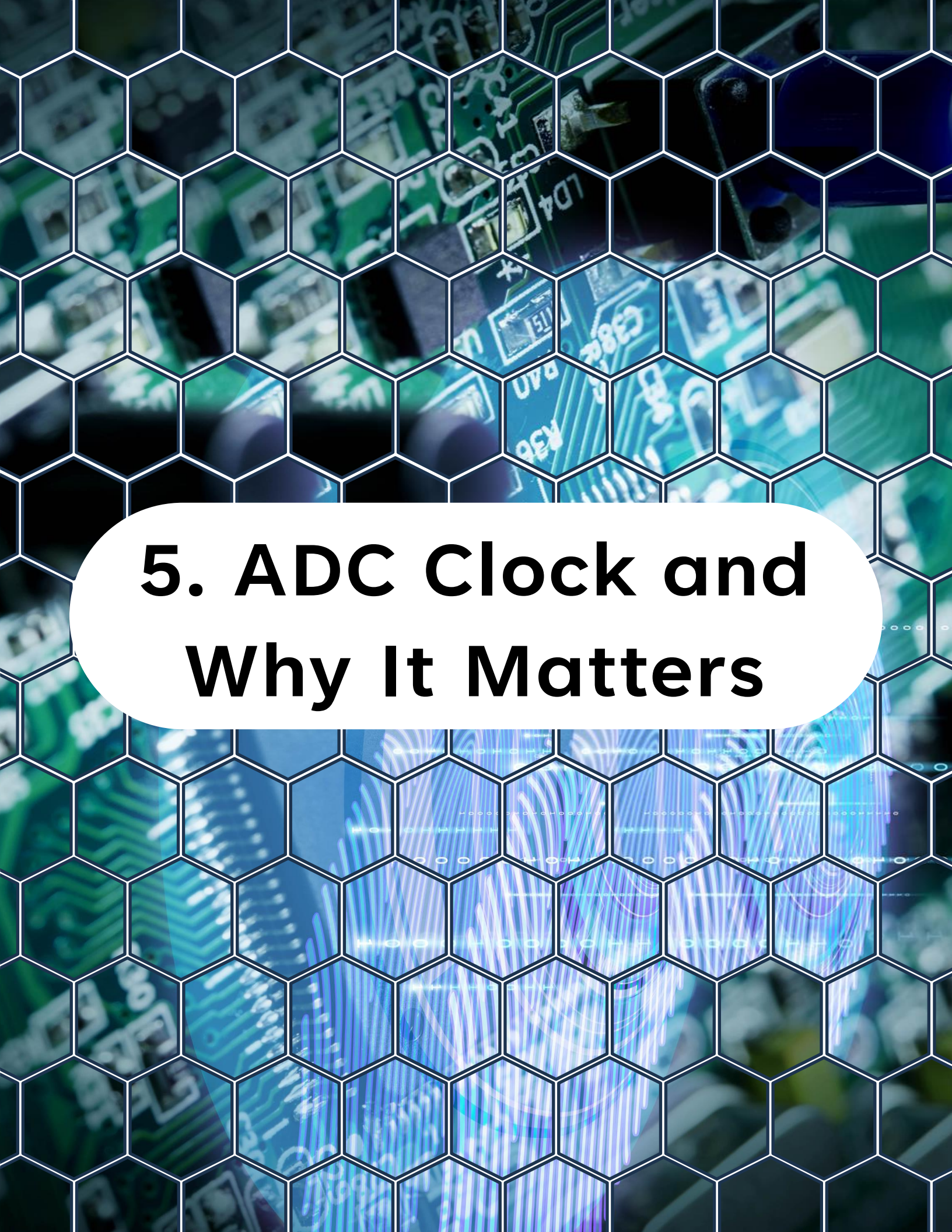
```
// Configure the ADC0 peripheral with a prescaler of 4
ADC0.CTRLC = ADC_PRESC_DIV4_gc | ADC_REFSEL_VDDREF_gc;

// Enable ADC0 reference AND SELECT VDD as reference
VREF.ADC0REF = 1 << VREF_ALWAYSON_bp | VREF_REFSEL_VDD_gc;

// Select the AIN0 pin as the positive input for the ADC conversion.
ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;

// Enable the ADC0 peripheral and start a conversion.
ADC0.CTRLA = ADC_ENABLE_bm;

```

4. Basic ADC Configuration Flow

Conceptual example:

```
// Configure the ADC0 peripheral with a prescaler of 4
ADC0.CTRLC = ADC_PRESC_DIV4_gc | ADC_REFSEL_VDDREF_gc;

// Enable ADC0 reference AND SELECT VDD as reference
VREF.ADC0REF = 1 << VREF_ALWAYSON_bp | VREF_REFSEL_VDD_gc;

// Select the AIN0 pin as the positive input for the ADC conversion.
ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;

// Enable the ADC0 peripheral and start a conversion.
ADC0.CTRLA = ADC_ENABLE_bm;

// Start a conversion by setting the ADC_STCONV bit
// in the ADC0.COMMAND register.
ADC0.COMMAND = ADC_STCONV_bm;

// Wait for the conversion to complete by polling the ADC_RESRDY bit
// in the ADC0.INTFLAGS register.
while (!(ADC0.INTFLAGS & ADC_RESRDY_bm));

// Read the conversion result and clear the interrupt flag
uint16_t result = ADC0.RES;
ADC0.INTFLAGS = ADC_RESRDY_bm;
```

That works. But it blocks the CPU. We can do better.



5. ADC Clock and Why It Matters

5. ADC Clock and Why It Matters

ADC accuracy depends on:

- ADC clock frequency
- Sampling time
- Input source impedance

If the ADC clock is too fast:

- Reduced accuracy
- Incomplete sampling
- Increased noise

If too slow:

- Slower measurements
- Reduced bandwidth

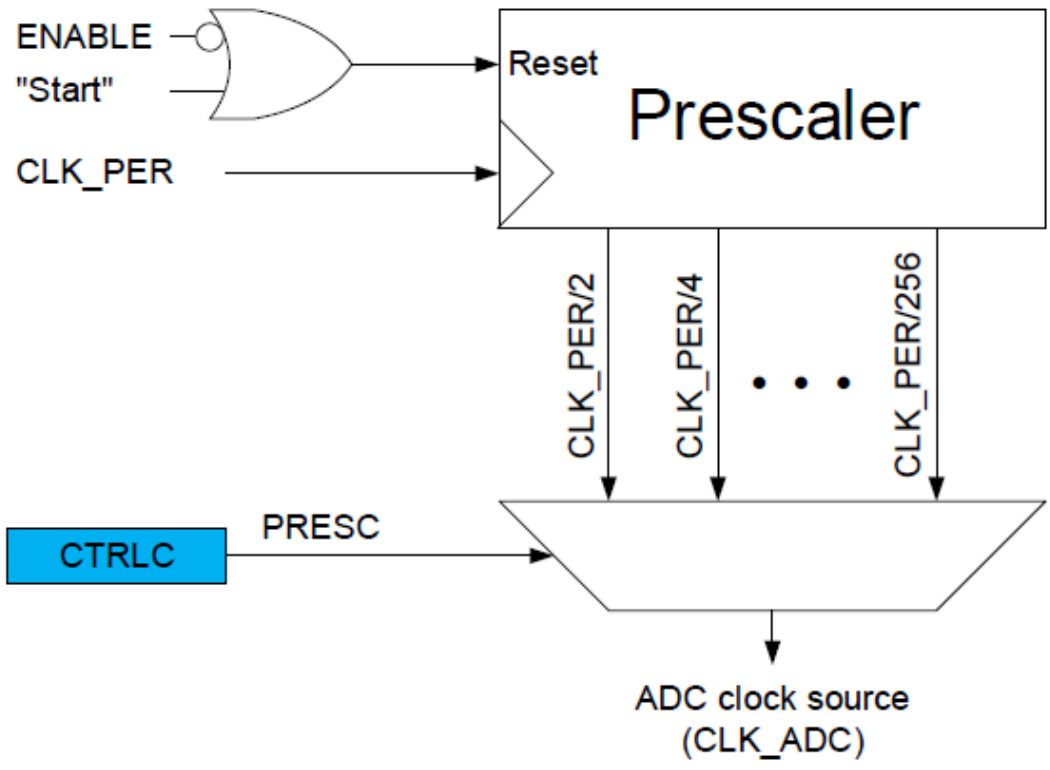
5. ADC Clock and Why It Matters

Typical design approach:

- Keep ADC clock in recommended datasheet range
- Ensure source impedance is low enough
- Add small RC filter if needed

This is where hardware and firmware meet.

ADC Prescaler



6. Noise – The Real Enemy

6. Noise - The Real Enemy

In real boards, ADC noise comes from:

- Switching regulators
- PWM edges
- Digital bus activity
- Poor grounding
- Long wires
- High source impedance

If you test ADC on a breadboard with PWM nearby, you will see it immediately.

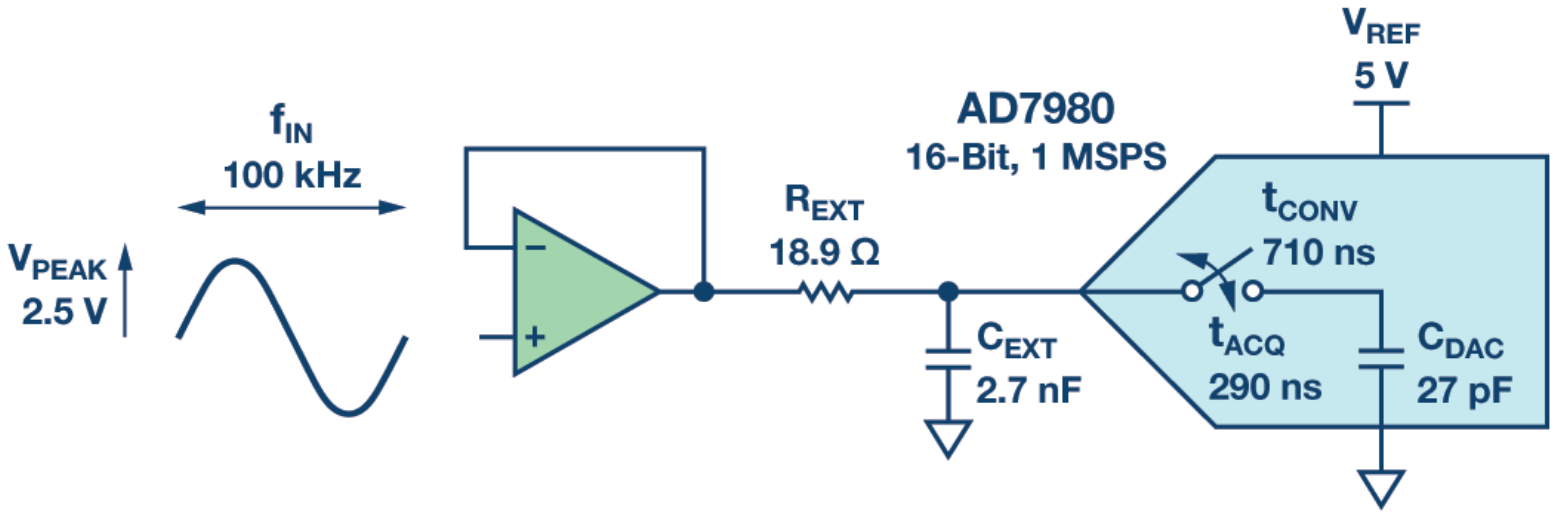
Solutions:

- Decoupling capacitors
- Ground planes
- RC filtering

6. Noise - The Real Enemy

- Averaging in firmware
- Hardware triggering synchronized with system timing

RC filter



7. Software Averaging

7. Software Averaging

The simplest noise reduction method:

Take N samples and average them.

Example:

```
uint32_t sum = 0;

// Take 16 samples and accumulate the results
for (uint8_t i = 0; i < 16; i++)
{
    // Start a conversion and wait for it to complete
    ADC0.COMMAND = ADC_STCONV_bm;

    // Wait for the conversion to complete
    while (!(ADC0.INTFLAGS & ADC_RESRDY_bm));

    // Read the result and accumulate it
    sum += ADC0.RES;
    ADC0.INTFLAGS = ADC_RESRDY_bm;
}

// Calculate the average of the 16 samples
uint16_t avg = (uint16_t)(sum / 16);
```

This reduces random noise but increases

latency.

8. Hardware Accumulation (Better Option)

8. Hardware Accumulation (Better Option)

The AVR DA ADC supports hardware accumulation.

Instead of manually summing:

- Configure accumulation (e.g., 16 samples)
- ADC hardware accumulates internally
- Result is automatically averaged (shifted)

Advantages:

- No CPU overhead
- Deterministic timing
- Cleaner code

This is usually preferred over manual averaging.

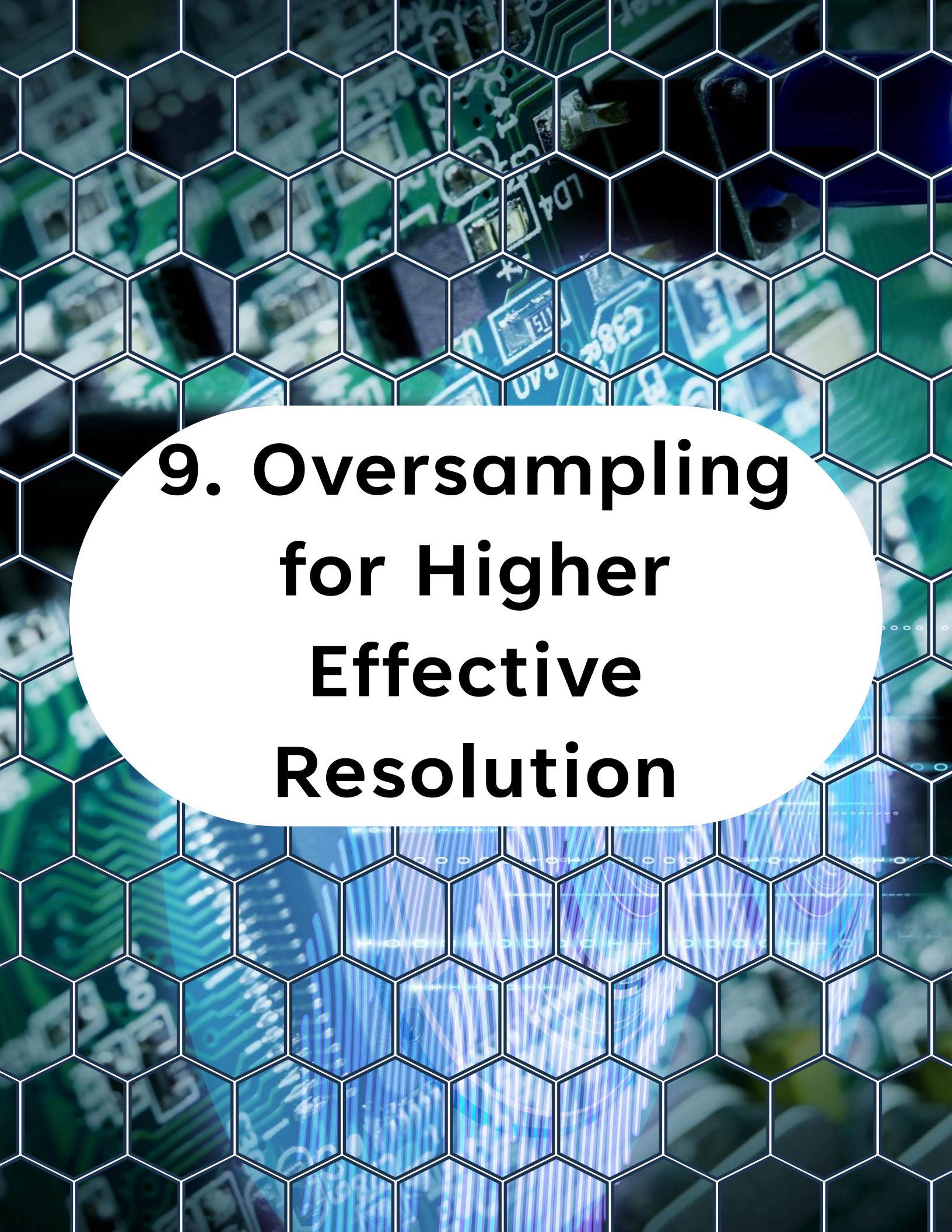
8. Hardware Accumulation (Better Option)

ADC Hardware Accumulation Example

```c
#include <avr/io.h>
#include <stdint.h>

#ifndef F_CPU
#define F_CPU (24000000UL)
#endif

#define ADC_INPUT_MUXPOS   ADC_MUXPOS_AIN0_gc

/* Choose accumulation samples: 16 samples is a good starter */
#define ADC_ACC_SAMPLES    16u

/**
 * @brief Initialize ADC0 for single-ended sampling with hardware accumulation.
 */
static void adc0_init_hw_accum(void)
{
    // Disable ADC before configuration
    ADC0.CTRLA = 0;

    // Configure the ADC clock prescaler (adjust as needed for your application)
    ADC0.CTRLC = ADC_PRESC_DIV4_gc;

    // Set reference voltage (adjust as needed; using VDD in this example)
    VREF.ADC0REF = VREF_REFSEL_VDD_gc;

    // Select input channel
    ADC0.MUXPOS = ADC_INPUT_MUXPOS;

    // Enable hardware accumulation.
    ADC0.CTRLB = ADC_SAMPNUM_ACC16_gc; /* Accumulate 16 samples */

    // Enable ADC
    ADC0.CTRLA = ADC_ENABLE_bm;
}

/**
 * @brief Perform one accumulated conversion and return the accumulated result.
```

8. Hardware Accumulation (Better Option)

```
39  *
40  * @return Accumulated ADC result (sum of N samples).
41  *
42  * How to compute average:
43  * - avg = accum / N
44  */
45 static uint16_t adc0_read_accumulated(void)
46 {
47     /* Start conversion */
48     ADC0.COMMAND = ADC_STCONV_bm;
49
50     /* Wait for result ready */
51     while ((ADC0.INTFLAGS & ADC_RESRDY_bm) == 0)
52     {
53         /* Busy wait (for demo only) */
54     }
55
56     /* Read result (accumulated) */
57     uint16_t accum = ADC0.RES;
58
59     /* Clear flag */
60     ADC0.INTFLAGS = ADC_RESRDY_bm;
61
62     return accum;
63 }
64
65 int main(void)
66 {
67     adc0_init_hw_accum();
68
69     while (1)
70     {
71         /* Read accumulated result */
72         uint16_t accum = adc0_read_accumulated();
73
74         /* Compute average for 16 samples (if using ACC16) */
75         uint16_t avg = (uint16_t)(accum / 16u);
76
77         /* Place breakpoints here to inspect accum and avg */
78         (void)avg;
79     }
80 }
```



9. Oversampling for Higher Effective Resolution

9. Oversampling for Higher Effective Resolution

Oversampling trick:

If you:

- Sample 4x -> gain ~1 extra bit
- Sample 16x -> gain ~2 extra bits
- Sample 64x -> gain ~3 extra bits

Then:

- Sum results
- Right-shift appropriately

This increases effective resolution if noise is present (noise acts as dithering).

It is widely used in sensor designs.

10. Free-Running Mode

10. Free-Running Mode

Instead of manually starting conversions, ADC can run continuously:

- Start once
- ADC keeps converting
- Each result triggers interrupt

Configuration concept:

- Enable FREERUN
- Enable RESRDY interrupt

ISR example pattern:

```c
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>

#ifndef F_CPU
#define F_CPU (24000000UL)
#endif

/* Global variable updated by ADC ISR */
```

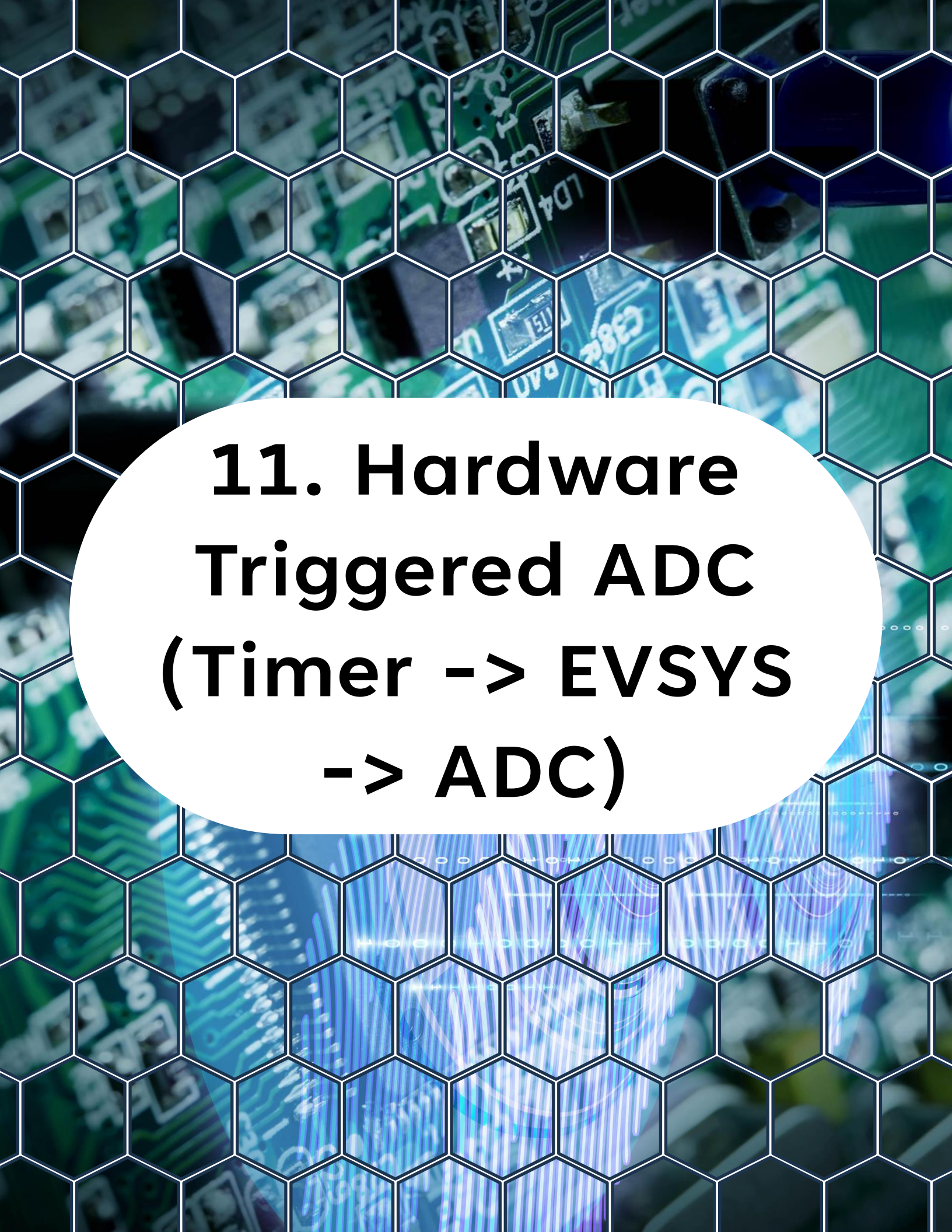
10. Free-Running Mode

```
/* Global variable updated by ADC ISR */
volatile uint16_t g_latest_adc = 0;

/**
 * @brief Initialize ADC0 in free-running mode on AIN0.
 */
static void adc0_init_freerun(void)
{
    /* Disable ADC before configuration */
    ADC0.CTRLA = 0;

    /* Select reference and prescaler */
    ADC0.CTRLC = ADC_REFSEL_VDDREF_gc | ADC_PRESC_DIV4_gc;

    /* Select input channel (example: AIN0) */
    ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;

    /* Enable free-running mode */
    ADC0.CTRLA |= ADC_FREERUN_bm;

    /* Enable Result Ready interrupt */
    ADC0.INTCTRL = ADC_RESRDY_bm;

    /* Clear any pending flag */
    ADC0.INTFLAGS = ADC_RESRDY_bm;

    /* Enable ADC */
    ADC0.CTRLA |= ADC_ENABLE_bm;

    /* Start first conversion (free-running continues automatically) */
    ADC0.COMMAND = ADC_STCONV_bm;
}

/**
 * @brief ADC0 Result Ready ISR.
 */
ISR(ADC0_RESRDY_vect)
```

10. Free-Running Mode

```
/**
 * @brief ADC0 Result Ready ISR.
 */
ISR(ADC0_RESRDY_vect)
{
    /* Read conversion result */
    g_latest_adc = ADC0.RES;

    /* Clear result ready flag */
    ADC0.INTFLAGS = ADC_RESRDY_bm;
}

int main(void)
{
    /* Initialize ADC in free-running mode */
    adc0_init_freerun();

    /* Enable global interrupts */
    sei();

    while (1)
    {
        /*
         * Main loop does not block on ADC.
         */

        uint16_t value = g_latest_adc;

        /* Example: convert to voltage (VDD reference assumed 5.0V) */
        float voltage = (value / 1023.0f) * 5.0f;

        (void)voltage;
    }
}
```

Now your main loop never blocks.



11. Hardware Triggered ADC (Timer -> EVSYS -> ADC)

11. Hardware Triggered ADC (Timer -> EVSYS -> ADC)

This is where things get professional.

Instead of:

"Start conversion whenever main loop feels like it“

You do:

- Timer generates periodic event
- **EVSYS** routes event to **ADC**
- **ADC** converts exactly at fixed intervals

Benefits:

- Deterministic sampling rate
- Perfect for control systems
- Reduced jitter
- Less CPU load

11. Hardware Triggered ADC (Timer -> EVSYS -> ADC)

Example architecture:

TCA overflow -> **EVSYS** -> **ADC** start conversion

Now your ADC runs at exactly 10 kHz or 1 kHz

or whatever you configured.

ADC Hardware Triggered Example

```c
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>

#ifndef F_CPU
#define F_CPU (24000000UL)
#endif

/* Desired sample rate (Hz) */
#define ADC_SAMPLE_HZ (1000UL)

/* Latest ADC sample, updated in ISR */
volatile uint16_t g_latest_adc_sample = 0;

/**
 * @brief Initialize TCA0 to overflow at ADC_SAMPLE_HZ and generate an event.
 */
static void tca0_init_overflow_event(uint32_t sample_hz)
{
    uint32_t top;

    /* Disable TCA0 before configuration */
    TCA0.SINGLE.CTRLA = 0;
```

11. Hardware Triggered ADC (Timer -> EVSYS -> ADC)

```
22      /* Disable TCA0 before configuration */
23      TCA0.SINGLE.CTRLA = 0;
24
25      /* Normal mode (default WGMODE = NORMAL) */
26      TCA0.SINGLE.CTRLB = 0;
27
28      /* Protect against divide by zero */
29      if (sample_hz == 0u) {
30          sample_hz = 1u;
31      }
32
33      /* Using DIV1 prescaler for simplicity */
34      top = (uint32_t)(F_CPU / sample_hz);
35      if (top != 0u) {
36          top -= 1u;
37      }
38
39      if (top > 0xFFFFu) {
40          top = 0xFFFFu;
41      }
42
43      /* Set period for overflow rate */
44      TCA0.SINGLE.PER = (uint16_t)top;
45
46      /* Enable timer */
47      TCA0.SINGLE.CTRLA = TCA_SINGLE_CLKSEL_DIV1_gc | TCA_SINGLE_ENABLE_bm;
48  }
49
50  /**
51   * @brief Configure EVSYS to route TCA0 overflow event to ADC0 start.
52   */
53  static void evsys_route_tca0_ovf_to_adc0(void)
54  {
55      /* Configure EVSYS Channel 0 generator = TCA0 overflow */
56      EVSYS.CHANNEL0 = (uint8_t)(EVSYS_CHANNEL0_TCA0_OVF_LUNF_gc);
57
58      /* Connect Channel 0 to ADC0 user */
59      EVSYS.USERADC0START = EVSYS_USER_CHANNEL0_gc;
60  }
61
62  /**
```

11. Hardware Triggered ADC (Timer -> EVSYS -> ADC)

```c
/**
 * @brief Initialize ADC0 to be triggered by EVSYS events (hardware triggered).
 */
static void adc0_init_event_triggered(void)
{
    /* Disable ADC before config */
    ADC0.CTRLA = 0;

    /* Select reference and prescaler */
    ADC0.CTRLC = ADC_REFSEL_VDDREF_gc | ADC_PRESC_DIV4_gc;

    /* Select input channel (example: AIN0) */
    ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;

    /* Enable "start on event" */
    ADC0.EVCTRL = ADC_STARTEI_bm;

    /* Enable result-ready interrupt */
    ADC0.INTCTRL = ADC_RESRDY_bm;

    /* Clear pending flag */
    ADC0.INTFLAGS = ADC_RESRDY_bm;

    /* Enable ADC */
    ADC0.CTRLA = ADC_ENABLE_bm;
}

/**
 * @brief ADC0 result-ready ISR.
 */
ISR(ADC0_RESRDY_vect)
{
    /* Read result */
    g_latest_adc_sample = ADC0.RES;

    /* Clear flag */
    ADC0.INTFLAGS = ADC_RESRDY_bm;
}

int main(void)
{
```

11. Hardware Triggered ADC (Timer -> EVSYS -> ADC)

```
100
101 int main(void)
102 {
103     /* 1) Configure timer to generate periodic overflow events */
104     tca0_init_overflow_event(ADC_SAMPLE_HZ);
105
106     /* 2) Route timer overflow event -> ADC start via EVSYS */
107     evsys_route_tca0_ovf_to_adc0();
108
109     /* 3) Configure ADC to start conversion on incoming event */
110     adc0_init_event_triggered();
111
112     /* Enable global interrupts (ADC ISR) */
113     sei();
114
115     while (1) {
116         /*
117          * Main loop is free. ADC sampling runs at
118          * ADC_SAMPLE_HZ in hardware.
119          */
120         uint16_t s = g_latest_adc_sample;
121         (void)s;
122     }
123 }
```

12. Why Hardware Triggering Is Superior

12. Why Hardware Triggering Is Superior

Polling-based sampling:

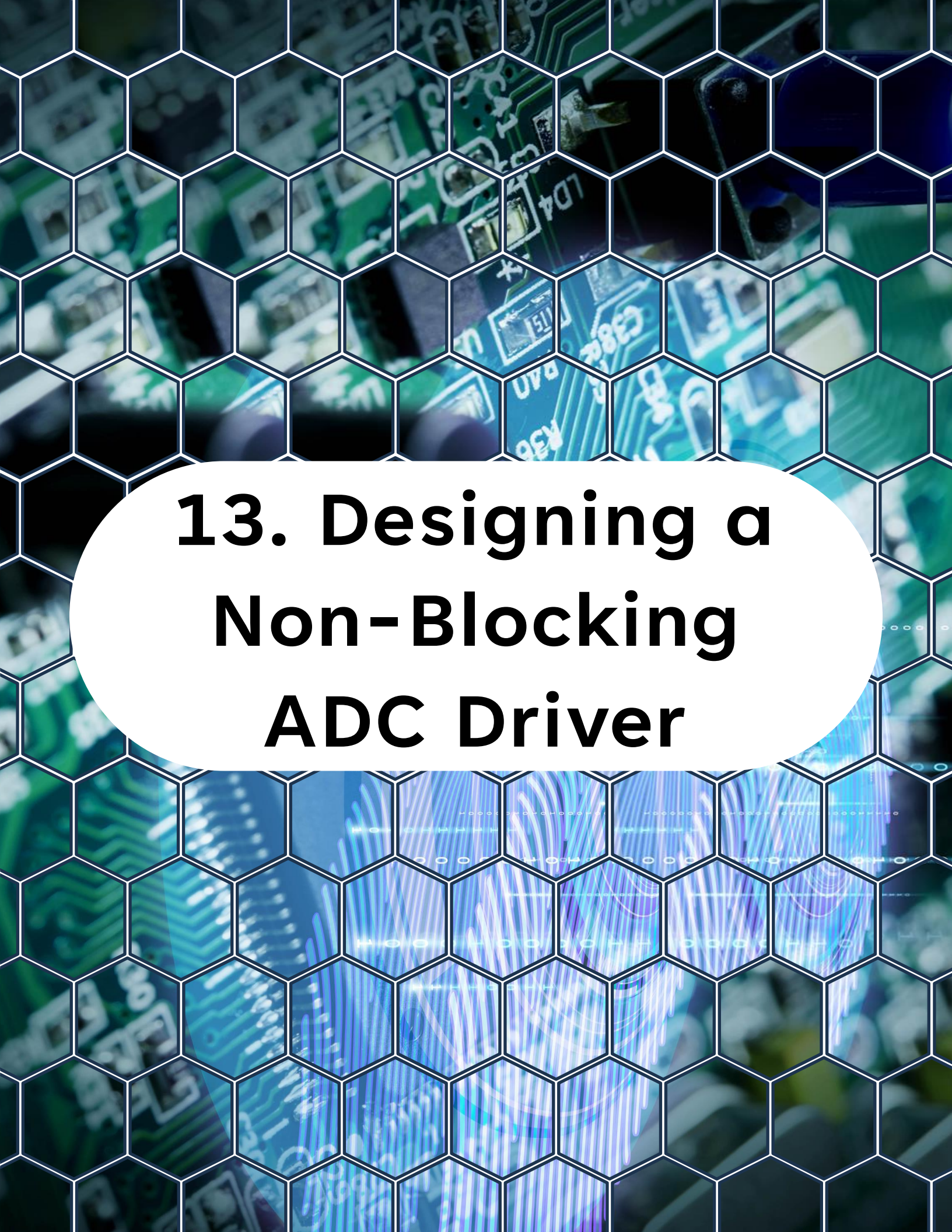
- Timing depends on code execution
- Jitter increases under load
- Hard to guarantee frequency

Event-triggered sampling:

- Hardware precision
- No jitter from CPU
- Cleaner signals
- Easier filtering

This is critical for:

- Motor control
- Power electronics
- Audio sampling
- Sensor fusion



13. Designing a Non-Blocking ADC Driver

13. Designing a Non-Blocking ADC Driver

A clean architecture:

Layer 1:

- adc_init()
- adc_start()
- adc_get_latest()

Layer 2:

- ISR stores result in ring buffer

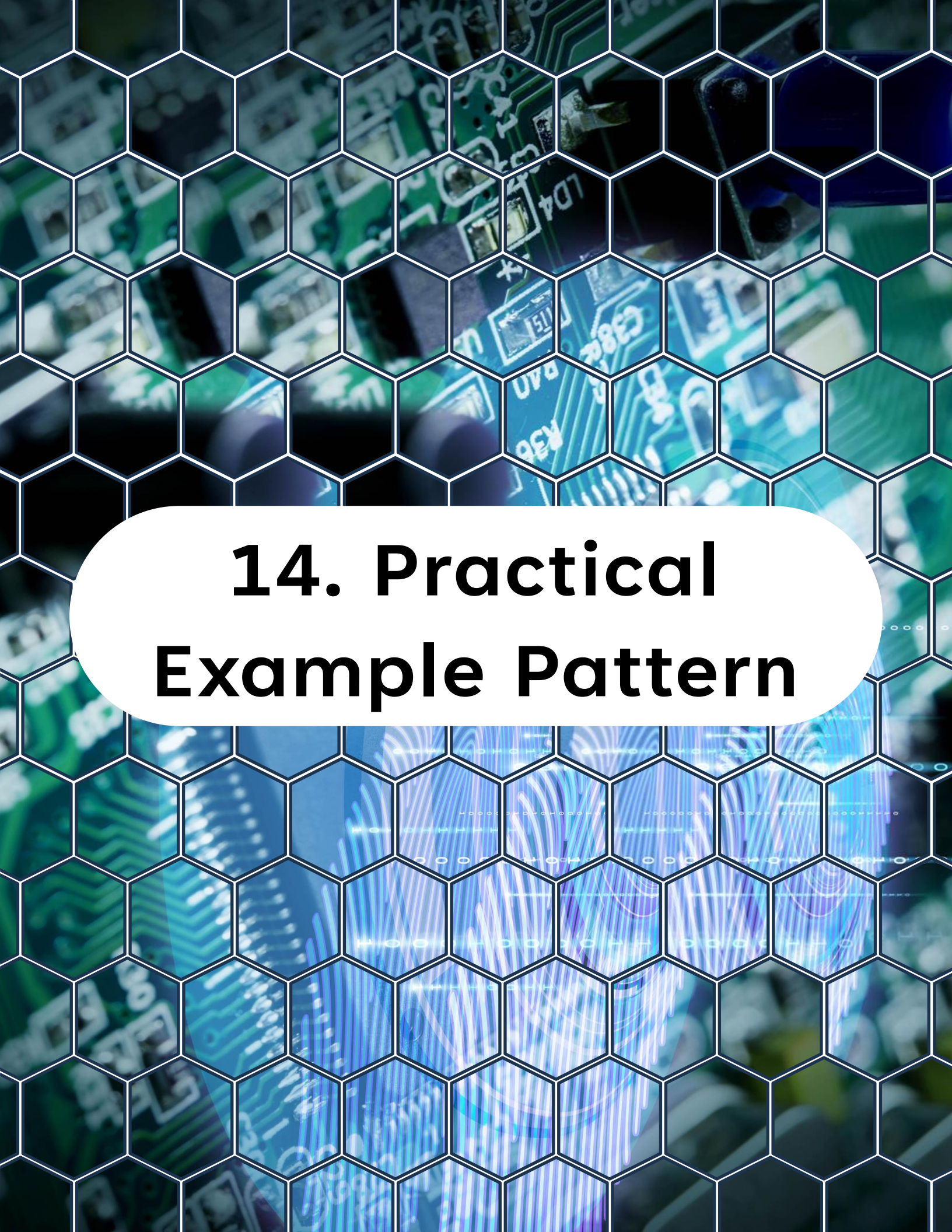
Layer 3:

- Application reads filtered result

Never make application code wait inside:

```
while (!(ADC0.INTFLAGS & ADC_RESRDY_bm));
```

That blocks everything.



14. Practical Example Pattern

14. Practical Example Pattern

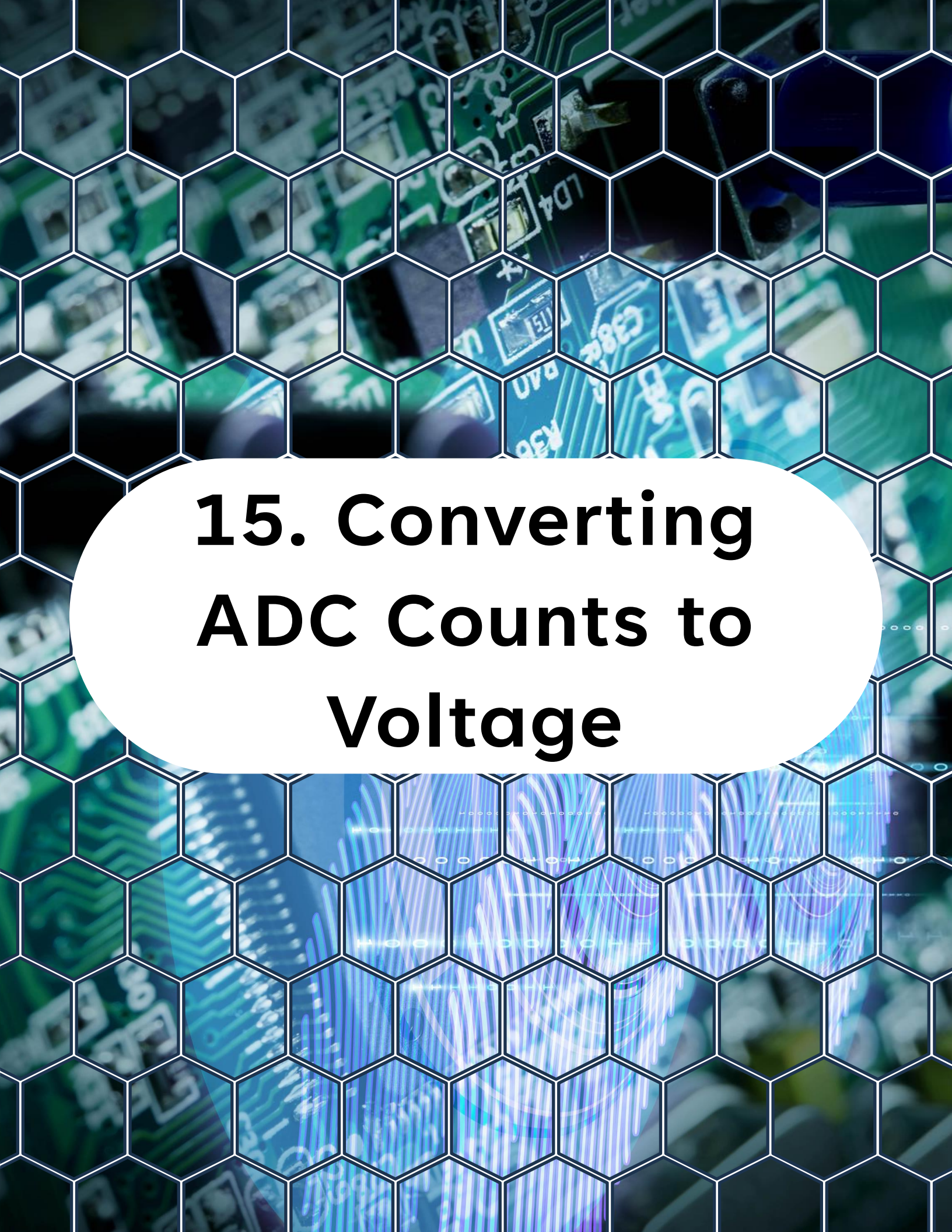
Use case:

Measure battery voltage every 1 ms.

Design:

- TCB0 generates 1 kHz event
- EVSYS routes to ADC
- ADC in single conversion mode
- ISR stores result
- Main loop processes averaged values

This pattern scales well.



15. Converting ADC Counts to Voltage

15. Converting ADC Counts to Voltage

If reference is VDD and resolution is 10-bit:

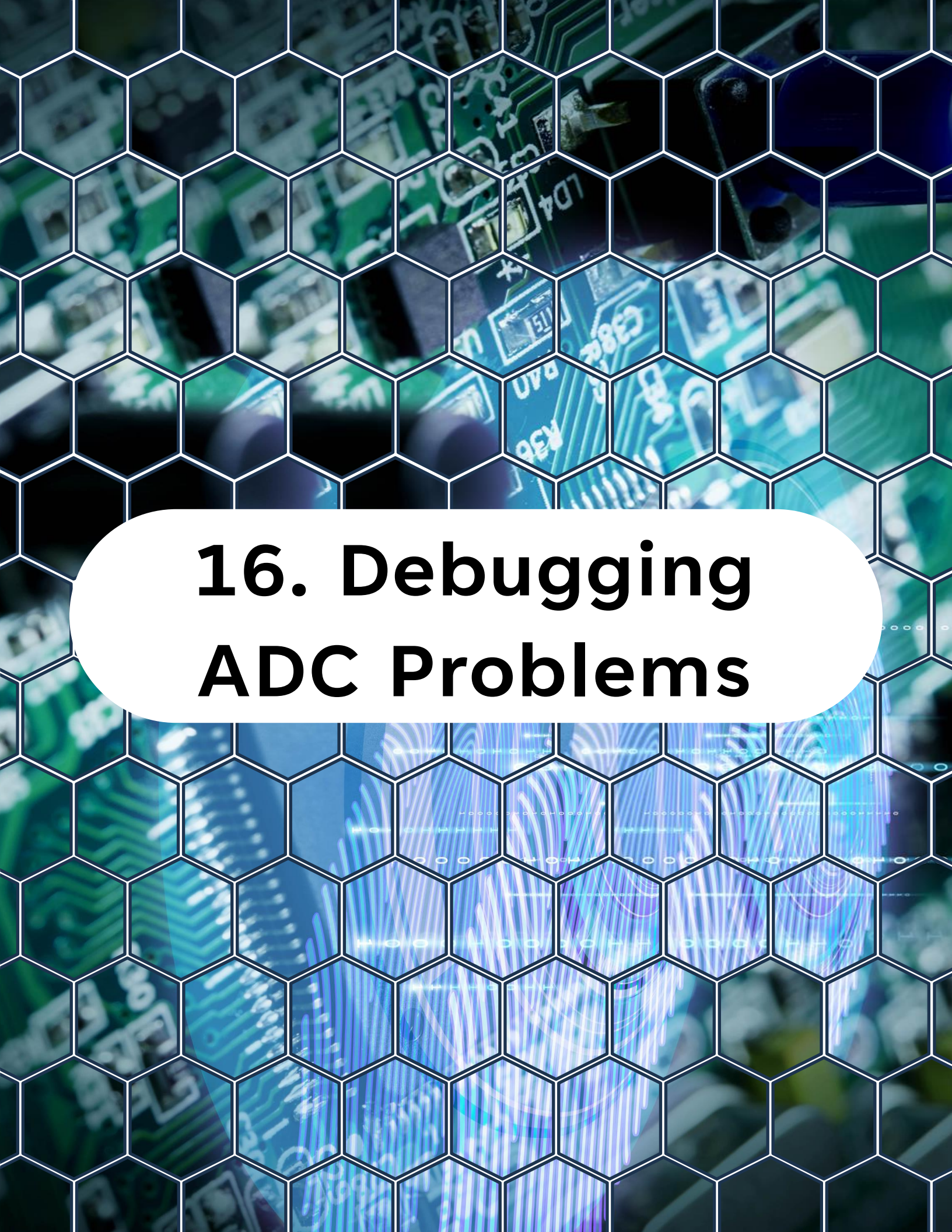
Voltage = (ADC_result / 1023.0) * VREF

If using 1.1V internal reference:

Voltage = (ADC_result / 1023.0) * 1.1V

Always verify which reference is actually selected.

Wrong reference assumption = wrong measurement.



16. Debugging ADC Problems

16. Debugging ADC Problems

If readings are unstable.

Check:

- Reference selection
- ADC clock speed
- Source impedance
- Grounding
- Nearby PWM activity
- Whether sampling occurs during switching edges

Synchronizing sampling to PWM off-time is a powerful technique.

17. What You Learned Today

17. What You Learned Today

You now understand:

- How to configure and use the **ADC** properly
- Why averaging matters
- Hardware vs software averaging
- Oversampling for higher resolution
- Event-triggered ADC conversions
- How to design non-blocking sampling architectures

You are no longer just "reading an analog pin".

You are designing measurement systems.

18. What Comes Next

18. What Comes Next

Day 19 - DAC: Signal Generation, Calibration, and Analog Control

Next you will:

- Generate analog voltages
- Create test signals
- Close the loop between ADC and DAC
- Build simple analog control systems

From here on, your firmware interacts deeply with the analog world.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course